Data Modeling

Ioana Manolescu
INRIA and Ecole Polytechnique



Outline

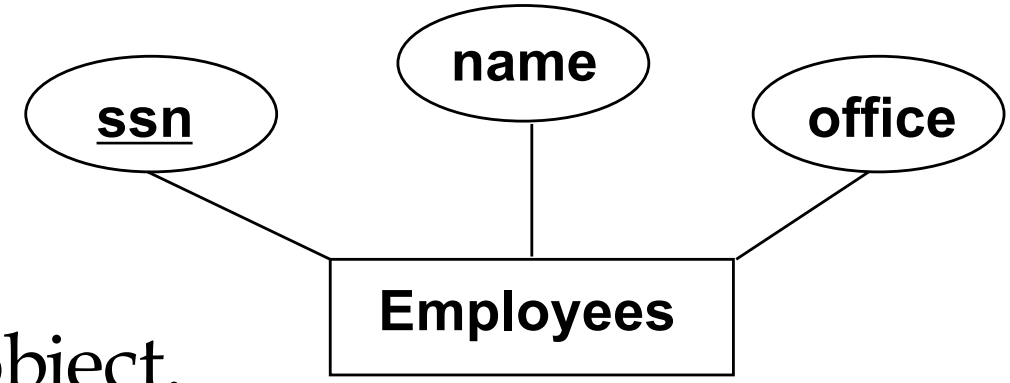
- Conceptual Design: ER model — *For humans*
- Relational Model — *For the DBMS*
- Logical Design: from ER to Relational — *Translation*

Conceptual Design

- Start with application requirements
- Use *Entity-Relationship (ER) Model*:
  - *Entities* and *relationships* in the enterprise
  - *Integrity constraints* (or business rules) that hold
  - ER diagrams pictorially represent all of the above.

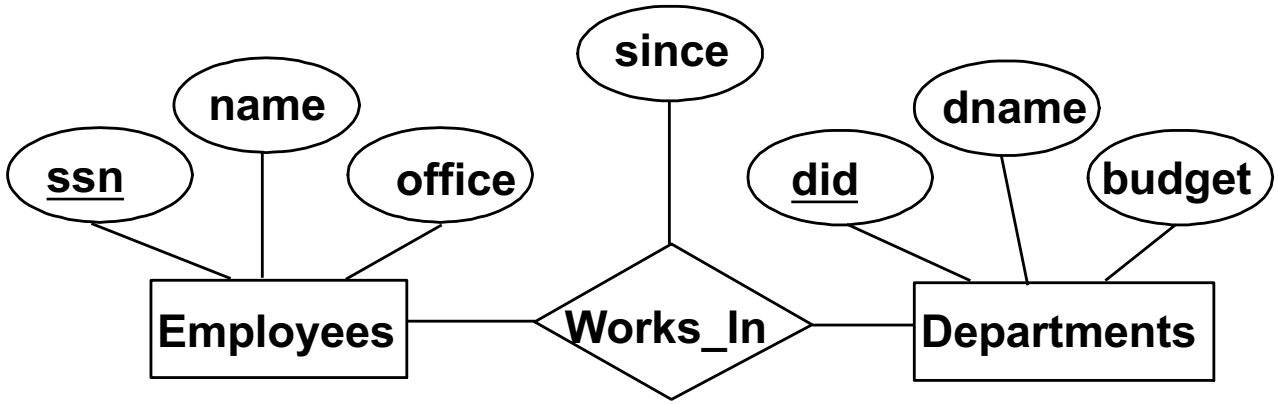
- When reading application requirements:
  - Entities, attributes are often extracted from *nouns*
  - Relationships are often from *verbs*

ER Model Basics: Entities



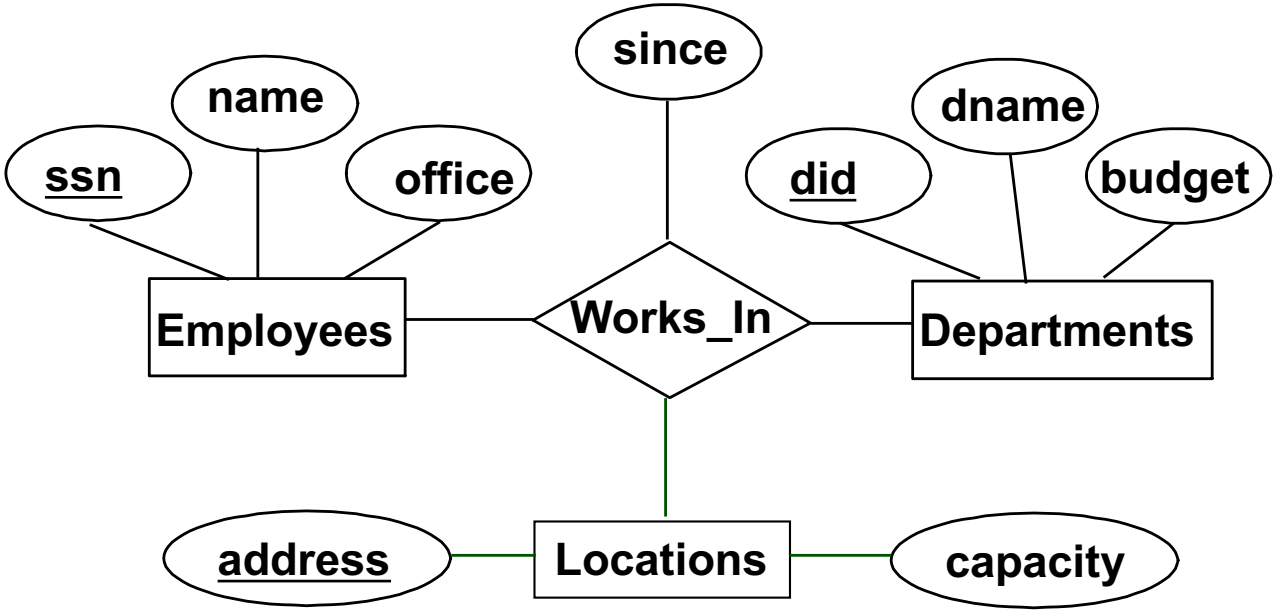
- *Entity:* A real-world object.
  - Described using a set of *attributes*.
- *Entity Set*: A collection of entities described by the same set of attributes.
  - *Domain* of an attribute.
  - *Key* of an entity set: **minimal** set of attributes that **uniquely** identify each entity in the set.

ER Model Basics: Relationships



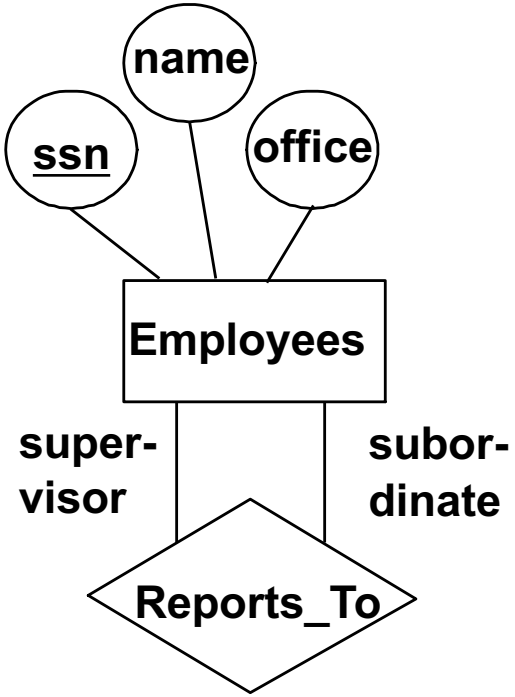
- *Relationship*: Association among 2+ entities.
  - E.g., Joe works in the accounting dept since 01/08.
- *Relationship Set*: Collection of similar relationships.

Ternary Relationships



- A **Works_In** relationship involves:
  - an employee
  - a department
  - a location

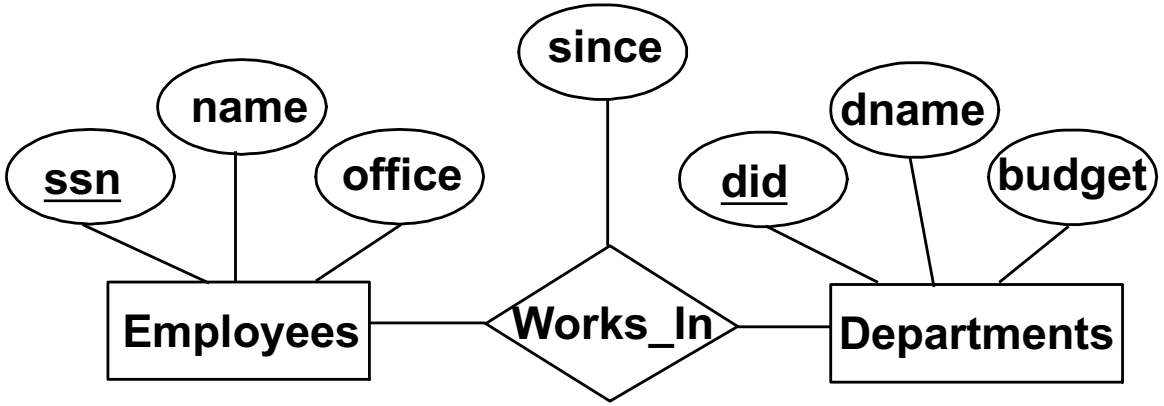
More on Relationships



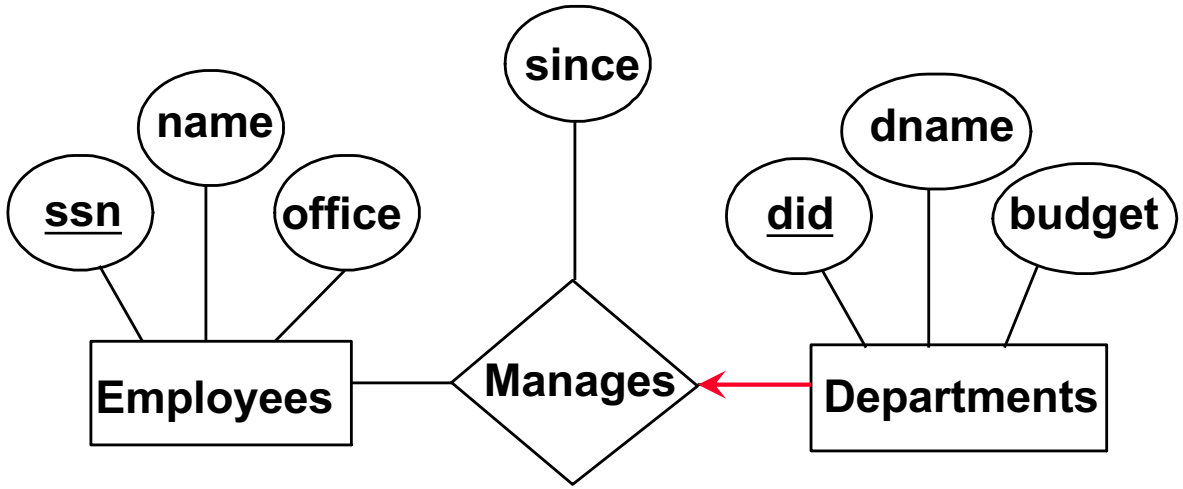
- An entity set can participate in same relationship set, but in **different roles**.

Key Constraints

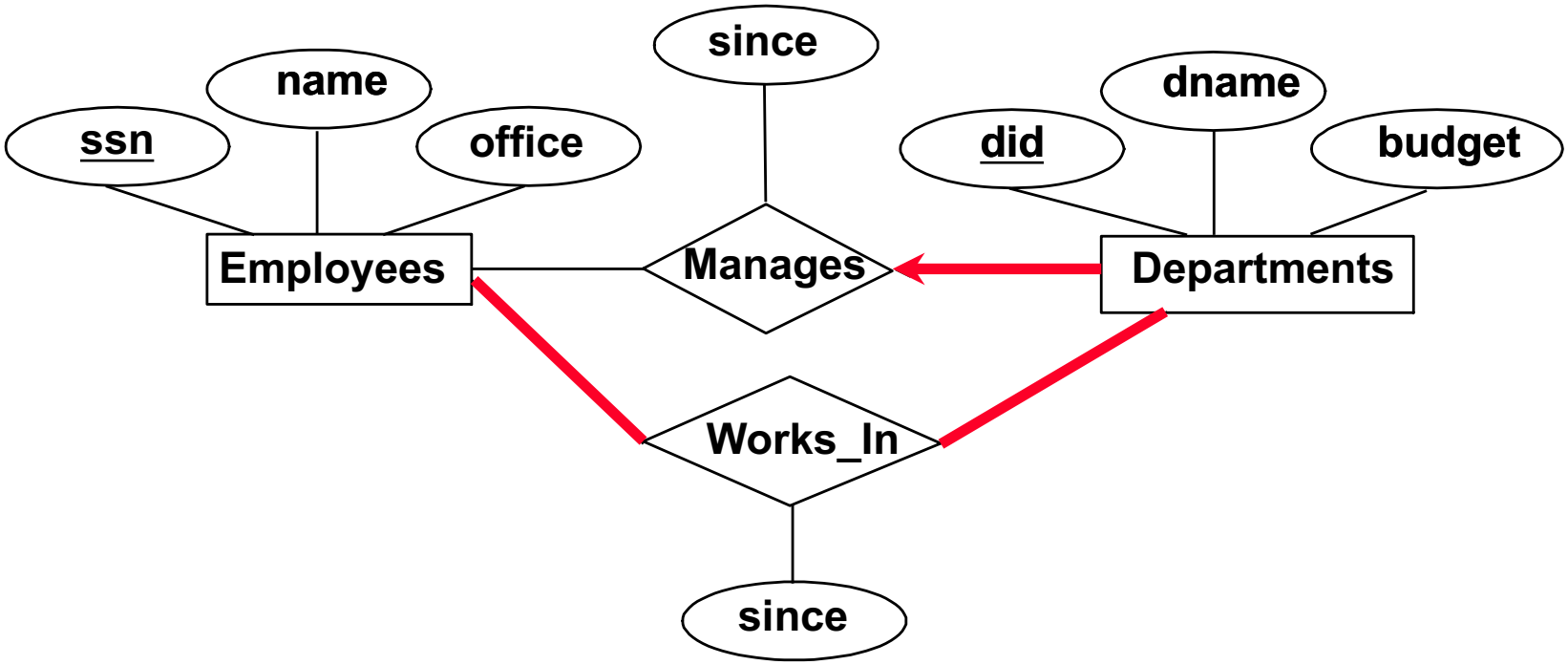
- **Works_In**: an employee can work in many depts; a dept can have many employees.
  - many-to-many (0:n)-(0,n)



- **Manages**: each dept has at most one manager
  - *key constraint* on Manages (←)
  - many-to-one (0:n)-(0:1)

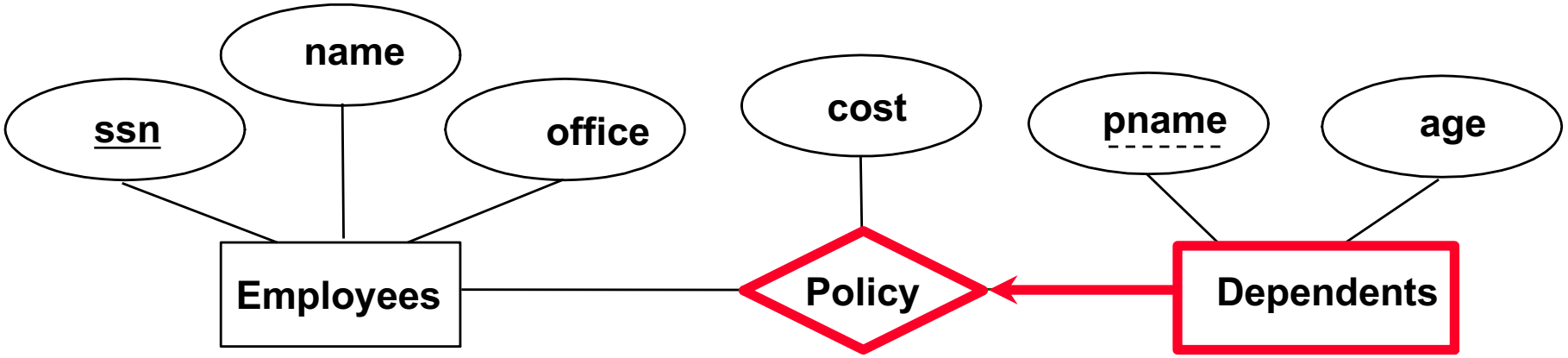


Participation Constraints



- **Works_In**: every employee works in at least one dept.
  - *Participation constraint* on Works_in, denoted by a **thick line,** (1:n)
  - Participation of Employees in Works_In is *total* (vs. *partial*).
- Key and participation constraints: exactly one

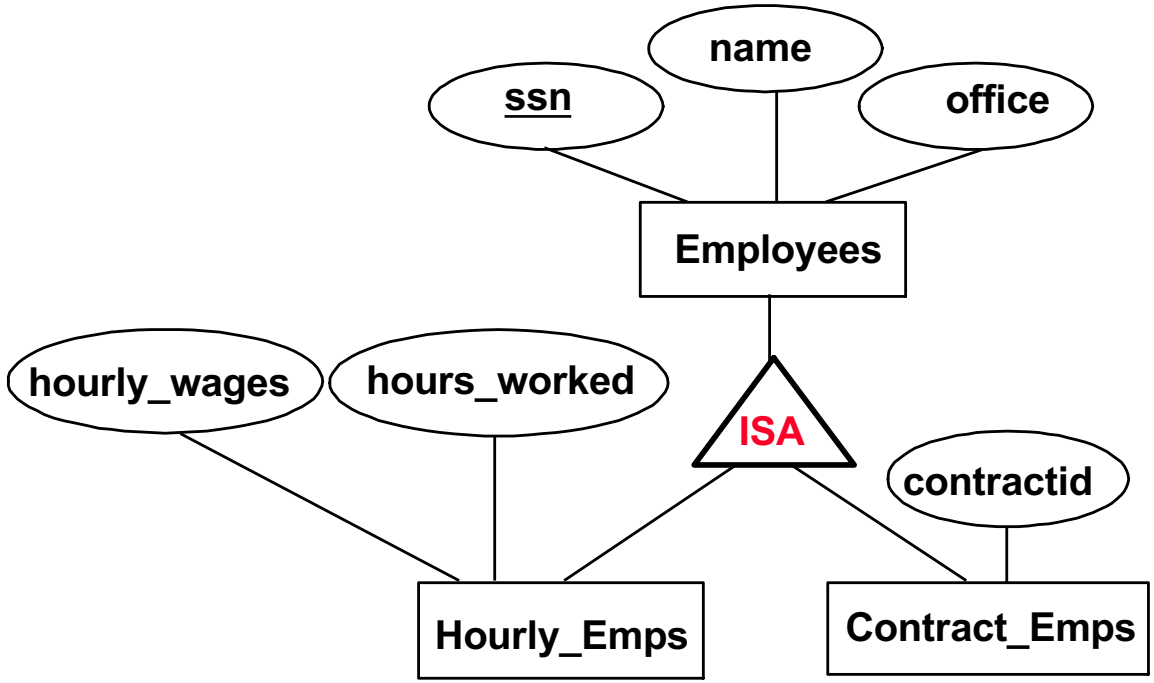
Weak Entities



Does an entity set always have a key?

- A *weak entity* can be identified uniquely only by considering the key of another (*owner*) entity.
  - A weak entity must have an exactly one relationship with its owner.
  - Notation: thick box highlighting the relationship which determines the entity

ISA ('is a') Hierarchies



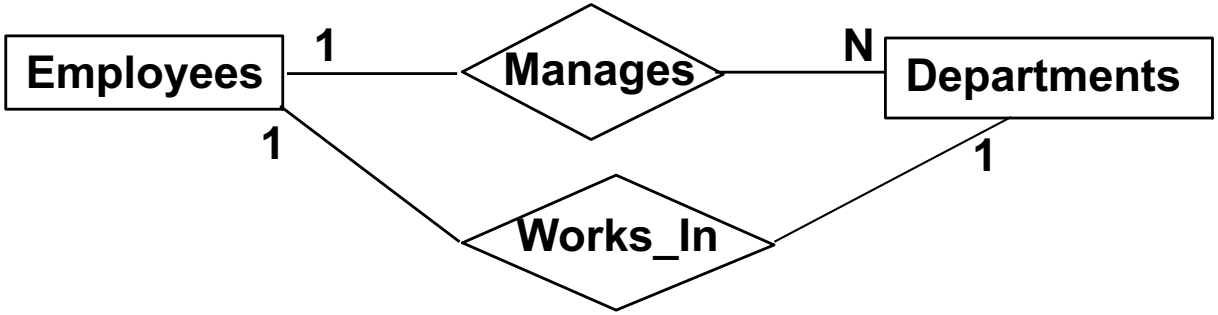
- It is sometimes natural to **classify** entities into subclasses.
- X *ISA* Y: every X entity is also considered to be a Y entity.
  - X entity set inherits all attributes of Y entity set.
  - X entity set may have its own descriptive attributes.
  - Queries asking for Y should also get X!

Issues with ISA Hierarchies

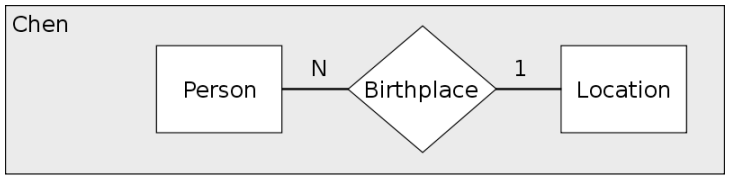
- *Overlap constraints*: Can Joe be an Hourly_Emps as well as a Contract_Emps entity?
  - *Allowed/disallowed (default: disallowed)*
    - *If allowed, write "OVERLAP"*
- *Covering constraints*: Does every Employees entity have to be an Hourly_Emps or a Contract_Emps entity?
  - *Yes/no (default: no)*
  - *If yes, write Hourly_Emps and ContractEmps COVERS Employees*
- Reasons for using ISA:
  - Add descriptive attributes specific to a subclass.
  - Identify entities that participate in a specific relationship.
    - Only SeniorEmps Manage; only Contract_Emps have Policies...

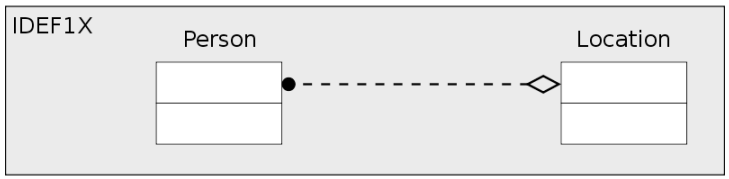
Alternative ER notations

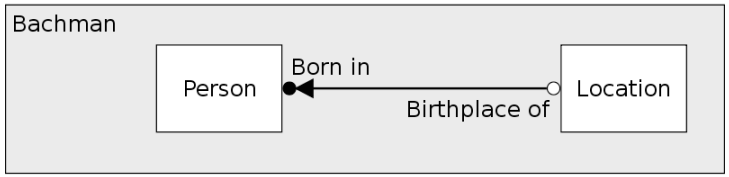
- Peter Chen notation:

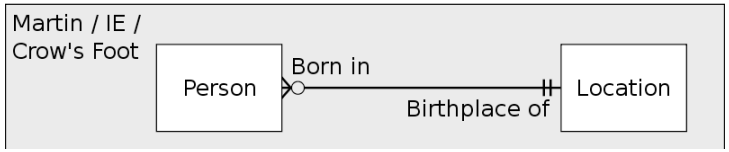


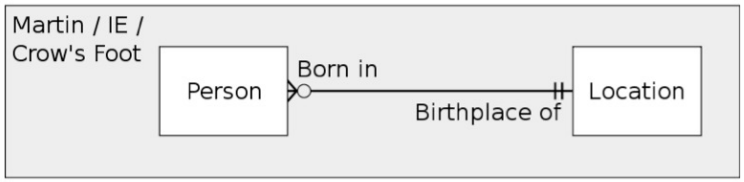
- Many alternatives exist. Also: Merise (FR)...

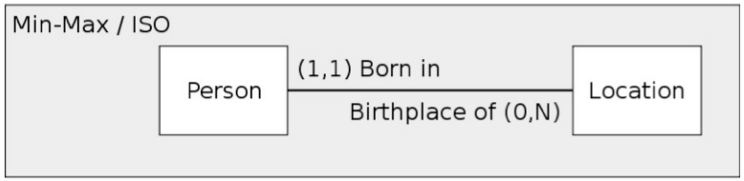


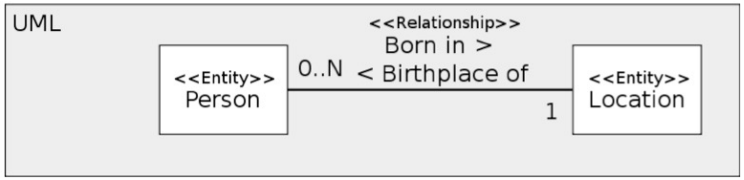












Outline

- Conceptual Design: ER model — *For humans*
- **Relational Model** — *For the DBMS*
- Logical Design: from ER to Relational — *Translation*

Relational Model

- A relational database is a set of *relations*.
- Each relation has:
  - *Schema* : specifies name of relation, name and type (domain) of each attribute.
    - Students(*sid*:string, *name*:string, *login*:string, *age*:integer, *gpa*:real).
  - *Instance* : a table with rows (*tuples*) and columns (*attributes, fields*).

    cardinality = #rows, *degree / arity* = #columns.
- A relation is a *set* of tuples (in theory).
  - All rows must be distinct, no duplicates.

Example Instance of Students Relation

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

- **Cardinality** = 3, **degree** = 5
- All rows are distinct.
- Some columns of two rows can be the same.

Creating Relations in SQL

- Create the Students relation

- Specify *domain constraints*:
  - type of each field
  - later enforced by the DBMS upon tuple insertion or update.

```
CREATE TABLE Students
        (sid     CHAR(20),
         name    CHAR(20),
         login   CHAR(10),
         age     INTEGER,
         gpa     REAL);
```

```
CREATE TABLE Enrolled
        (sid     CHAR(20),
         cid     CHAR(20),
         grade   CHAR(2));
```

Adding Tuples

- Can insert a single tuple using:

```
INSERT INTO Students (sid, name, login, age, gpa)
VALUES ('53688', 'Smith', 'smith@ee', 18, 3.2);
```

```
INSERT INTO Students
VALUES ('53688', 'Smith', 'smith@ee', 18, 3.2);
```

☛ *More efficient alternatives for bulk insertion*

Integrity Constraints

- *Integrity Constraints* (ICs): condition that must be true for **any** instance of the database.
  - *Domain constraint*
  - *Primary key constraint*
  - *Foreign key constraint*
  - Specified when the schema is defined.
- DBMS enforces ICs.
  - This ensures stored data is faithful to real-world meaning.
  - Avoids data entry errors, too

Primary Key Constraints

- *Key* of a relation: minimal set of attributes that uniquely identify each tuple.
  1. **No two tuples can have same values in all key fields**.
  2. This is **not true for any subset** of the key.

  - If 2. is false, we have a *superkey*.
  - If more than 1 key for a relation: *candidate keys*.
  - One of candidate keys is chosen to be the *primary key*.
- E.g., Students(sid, name, login, age, gpa)
- E.g., Enrolled(sid, cid, grade)

Primary and Candidate Keys in SQL

- Specify *candidate keys* using UNIQUE.
- Choose one candidate key as the *primary key*.

"For a given student and course, there is a single grade."

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid  CHAR(20),
   grade CHAR(2),
   PRIMARY KEY  (sid,cid));
```

"… and no two students in a course receive the same grade."

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid  CHAR(20),
   grade CHAR(2),
   PRIMARY KEY  (sid,cid),
   UNIQUE (cid, grade) );
```

Foreign Keys

- *Foreign key*: set of fields of a relation used to "refer" to the primary key of another relation.
  - Like a "logical pointer".
- E.g., Enrolled(*sid*: string, *cid*: string, *grade*: string):
  - *sid* is a foreign key referring to Students.

Foreign Keys in SQL

- Only students listed in the Students relation should be allowed to enroll for courses.

```
CREATE TABLE Enrolled
  (sid CHAR(20),  cid CHAR(20),  grade CHAR(2),
   PRIMARY KEY  (sid,cid),
   FOREIGN KEY (sid) REFERENCES Students );
```

Enrolled

sid	cid	grade
53666	Carnatic101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Referential Integrity

- *Referential integrity*: any foreign key value must have a matching primary key value in referenced relation.
  - E.g., every *sid* value in Enrolled must appear in Students.
  - No dangling references.

- Can you name a data management context where referential integrity is not enforced?

Enforcing Referential Integrity

- What if an Enrolled tuple with a non-existent student id is inserted?
  - *Reject it!*
- What if a Students tuple is deleted?
  - CASCADE: delete all Enrolled tuples that refer to it.
  - NO ACTION: disallow if the Students tuple is referred to.
    - NO ACTION is the default behavior
  - SET DEFAULT: set the foreign key to a *default sid*.
  - SET NULL: set the foreign key to a special value *null*, denoting '*unknown*' or '*inapplicable*'.
- Updates to *sid* in Students are treated similarly.

Referential Integrity in SQL

```
CREATE TABLE Enrolled
   (sid CHAR(20),
    cid CHAR(20),
    grade CHAR(2),
    PRIMARY KEY  (sid,cid),
    FOREIGN KEY (sid)
      REFERENCES Students
            ON DELETE CASCADE
            ON UPDATE NO ACTION);
```

Where do ICs Come From?

- Real-world business logic.
- Can check violation in a database instance, but can NEVER infer an integrity constraint by looking at an instance.
  - An integrity constraint is a statement about *all possible* instances!
  - E.g., *name* of the Students relation.

Views

- A *view* is just a relation, but we store a *definition*, rather than a set of tuples.

```
CREATE VIEW YoungActiveStudents (name, grade)
        AS SELECT S.name, E.grade
        FROM Students S, Enrolled E
        WHERE S.sid = E.sid and S.age<21
```

- Views can be dropped using the DROP VIEW command.
  - How to handle DROP TABLE if there's a view on the table?
    - DROP TABLE command has options to let the user specify: RESTRICT will fail if views; CASCADE erases all.

Views and Security

- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).
  - Given YoungActiveStudents, but not Students or Enrolled, we can find the names of these students, but not the *cid's* of the courses they are enrolled in.

Views are a core mechanisms for ensuring **privacy-preserving data sharing** through an RDBMS

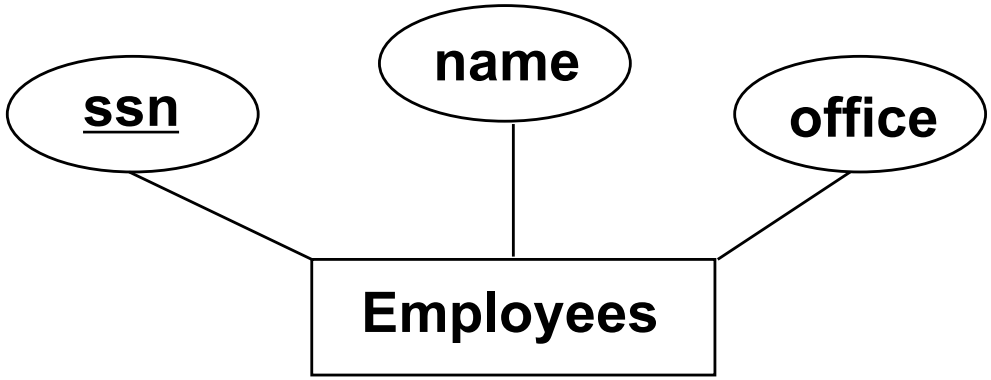
Another such mechanism are **SQL access rights** (to be seen shortly)

Outline

- Conceptual Design: ER model — *For humans*
- Relational Model — *For the DBMS*
- **Logical Design: from ER to Relational** — *Translation*

Logical DB Design: ER to Relational

- An entity set is translated to a table.




```
CREATE TABLE Employees
        (ssn CHAR(11),
         name CHAR(20),
         office  INTEGER,
         PRIMARY KEY  (ssn));
```

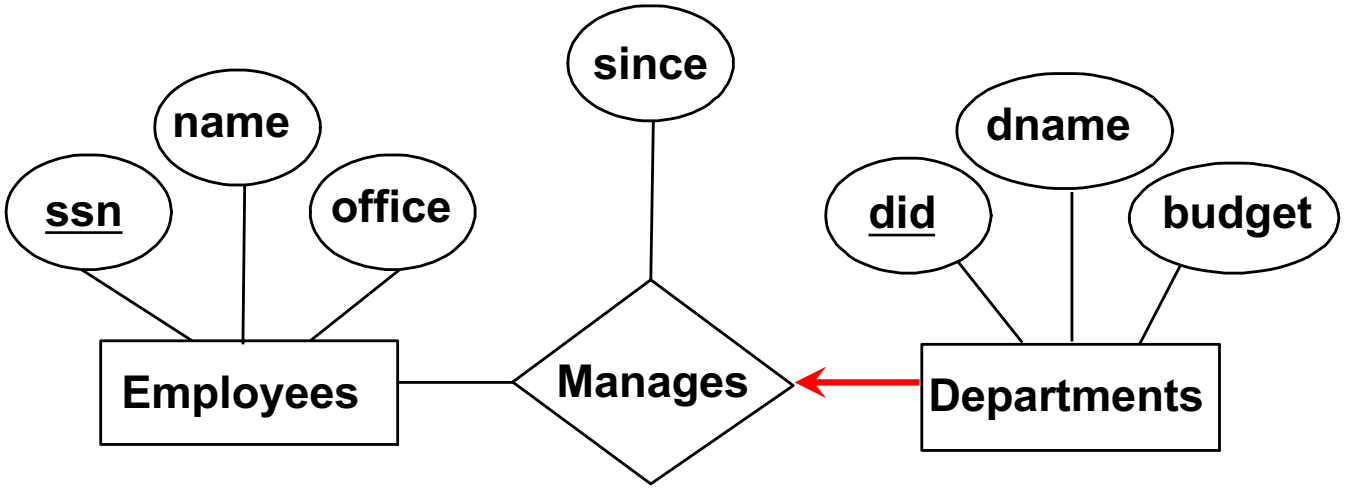
Relationship Sets to Tables

- A relationship set is also translated to a table.
- Attributes must include:
  - all descriptive attributes;
  - primary key of each related entity set, as a *foreign key*.
- All foreign keys are part of a *superkey* of the relation table.

```
CREATE TABLE Works_In(
  ssn  CHAR(11),
  did  INTEGER,
  since  DATE,
  PRIMARY KEY (ssn, did),
  FOREIGN KEY (ssn)
      REFERENCES Employees,
  FOREIGN KEY (did)
      REFERENCES Departments);
```

Review: Key Constraints

- *Key constraint:* Each dept has *at most one* manager.

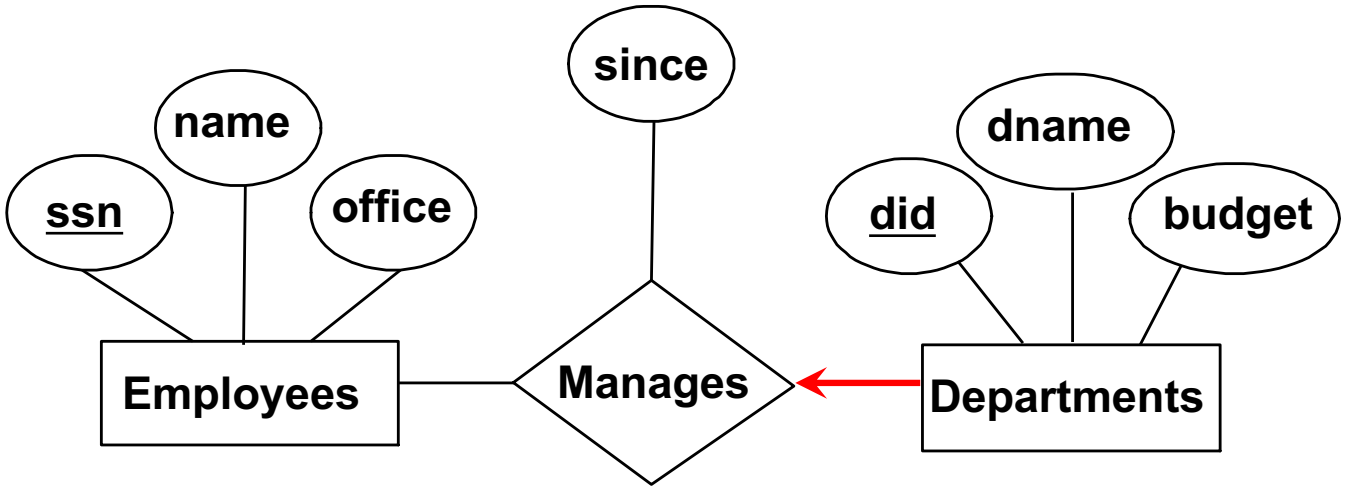


Translating ER Diagrams w. Key Constraints

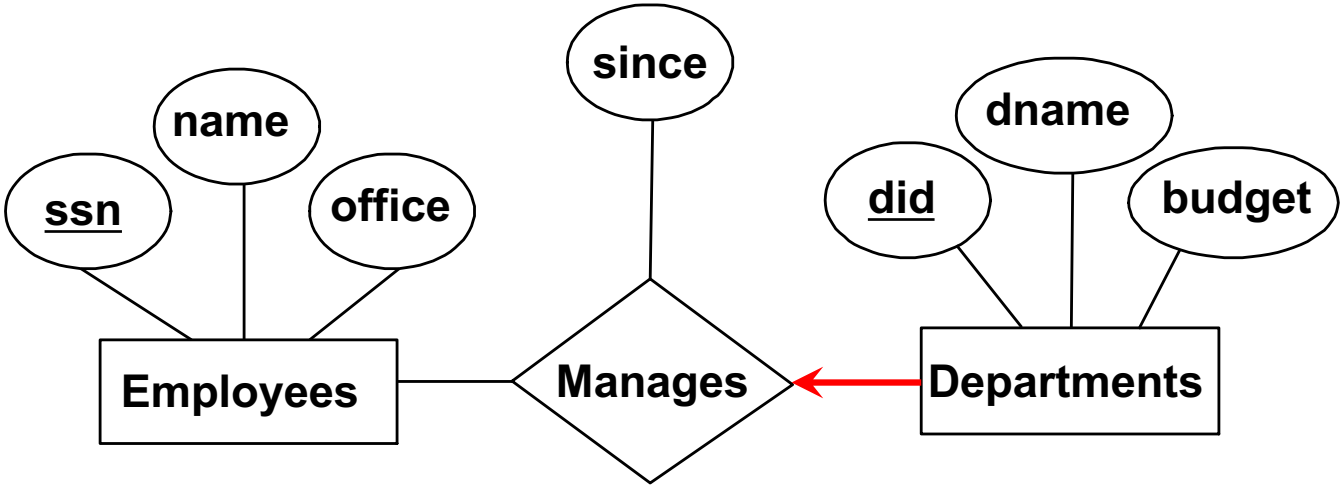
- Two approaches

1. A separate table for Manages:
   - Borrow primary key from the entity with the key constraint.

```
CREATE TABLE Manages(
  ssn CHAR(11),
  did INTEGER,
  since DATE,
  PRIMARY KEY (did),
  FOREIGN KEY (ssn) REFERENCES Employees,
  FOREIGN KEY (did) REFERENCES Departments);
```



Translating ER Diagrams w. Key Constraints

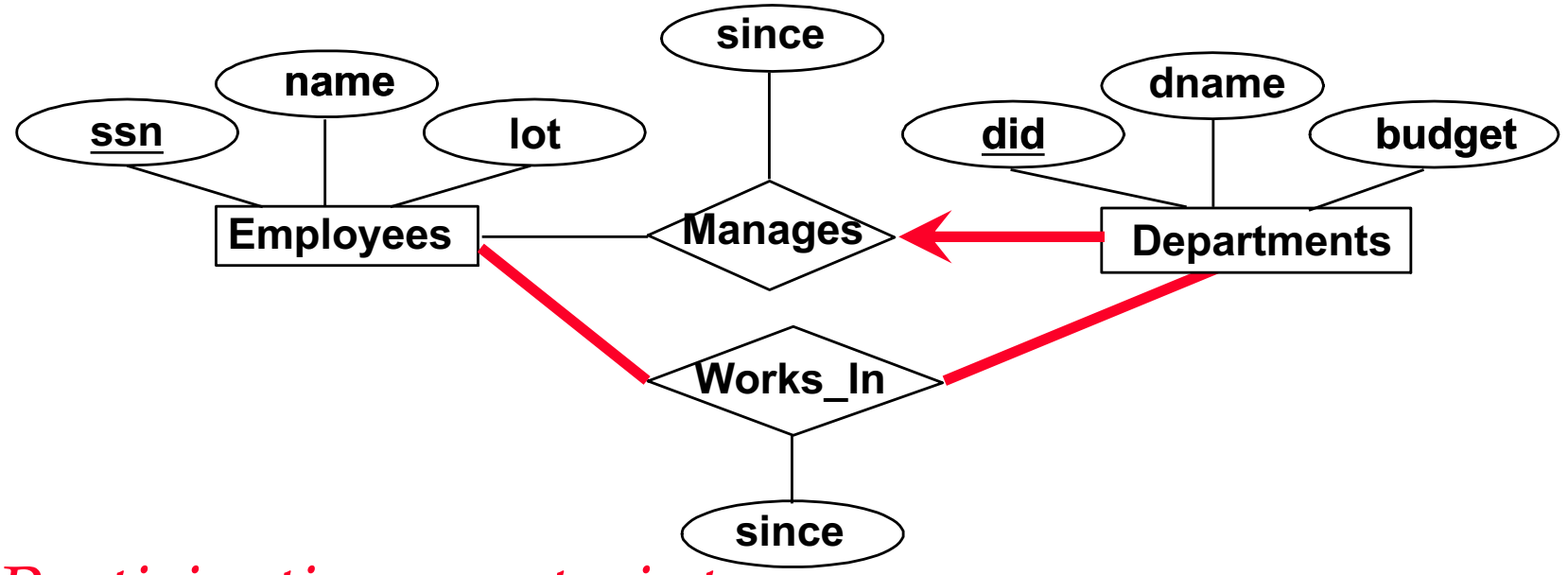


2. Merge the relationship into the table of the entity with the key constraint.

- In our example, Manages into Department

```
CREATE TABLE Dept_Mgr(
  did INTEGER,
  dname CHAR(20),
  budget REAL,
  ssn CHAR(11),
  since DATE,
  PRIMARY KEY (did),
  FOREIGN KEY (ssn) REFERENCES Employees);
```

Review: Participation Constraints



- *Participation constraint:*
  - Every employee works in at least one department.
  - Every department has at least one employee.
  - Every department must have one manager.

Translating Key and Participation Constraints

- If we have both key and participation constraints:

```
CREATE TABLE  Dept_Mgr(
   did  INTEGER,
   dname  CHAR(20),
   budget  REAL,
   ssn  CHAR(11) NOT NULL,
   since  DATE,
   PRIMARY KEY  (did),
   FOREIGN KEY  (ssn) REFERENCES Employees,
      ON DELETE NO ACTION
      ON UPDATE CASCADE);
```

Translating Cyclic Participation Constraints

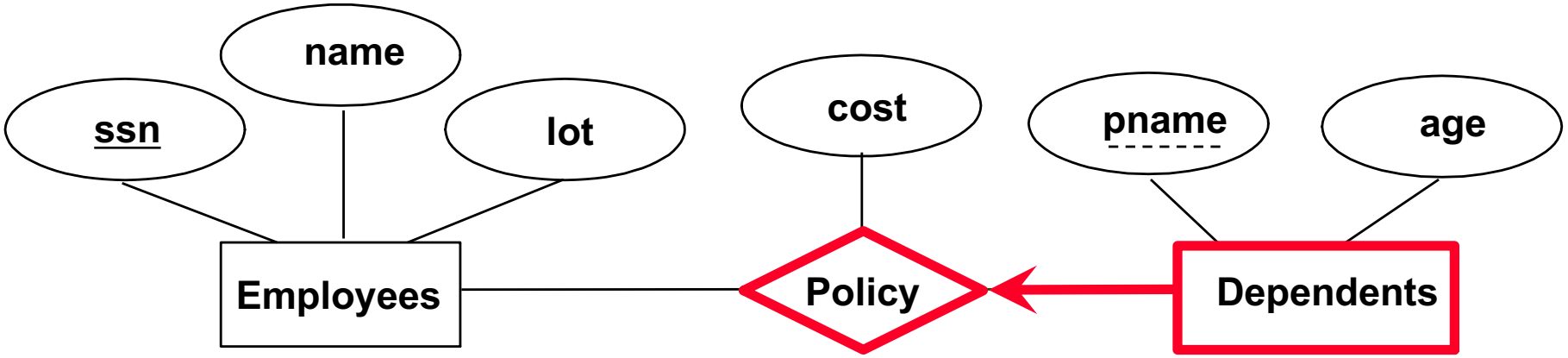
- E.g., every department must have a manager, and every manager must have a department
- Creating the tables:
  - CREATE TABLE **Manager**... mid, did... PRIMARY KEY (mid);
  - CREATE TABLE **Department**... PRIMARY KEY (did), FOREIGN KEY (mid) REFERENCES Manager;
  - ALTER TABLE **Manager** ADD FOREIGN KEY (mid) REFERENCES Department;
- Populating the tables:
  - **BEGIN TRANSACTION DEFERRABLE**;
  - INSERT INTO Manager VALUES(...);
    INSERT INTO Department VALUES(...);
  - **END TRANSACTION;**
  - The constraints are verified only at the end

Participation Constraints Only

- For participation constraints only, need to resort to *checks* (*ICs over a single table)* or *assertions (ICs over several tables*).

```
ALTER TABLE Employees ADD CONSTRAINT
participation
CHECK ((SELECT COUNT(*)
        FROM Works_In
        WHERE Works_In.ssn = Employees.ssn)  > 0);
```

Review: Weak Entities



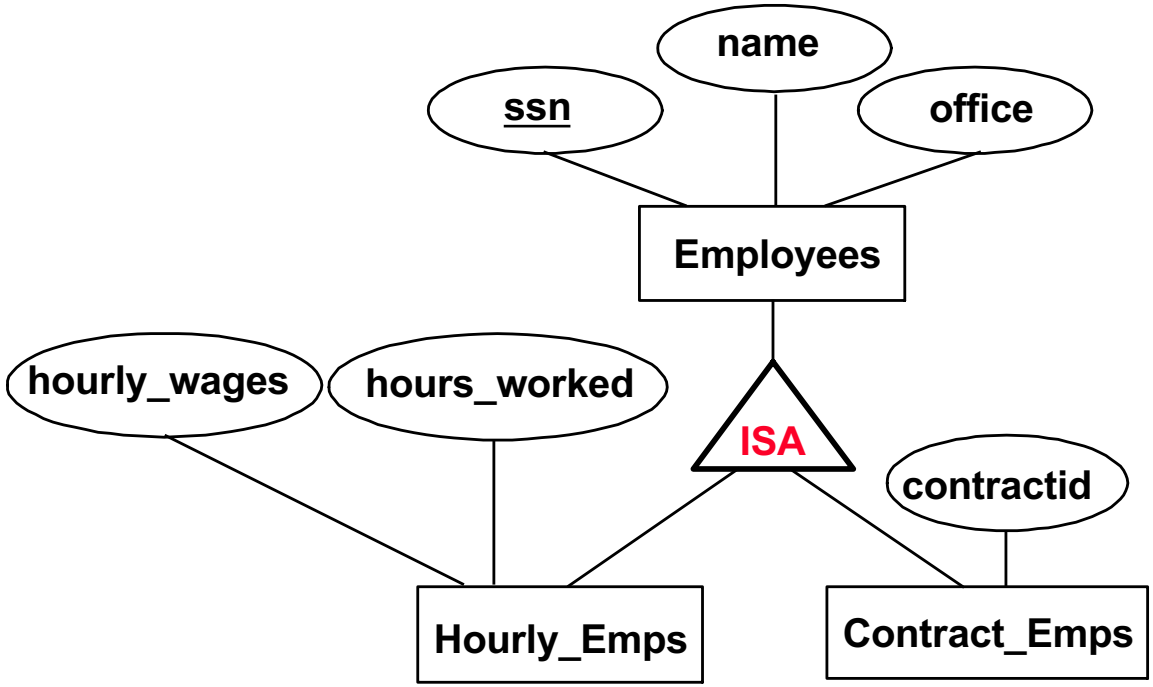
- A *weak entity* can be identified uniquely only by considering the primary key of another (*owner*) entity.
  - A weak entity must have an exactly one relationship with its owner.

Translating Weak Entity Sets

- Merge *weak entities* and *identifying relationships* into a single table.
  - When the *owner entity* is deleted, all related weak entities must also be deleted.

```
CREATE TABLE  Depndt_Policy (
  pname          CHAR(20),
  age    INTEGER,
  cost   REAL,
  ssn    CHAR(11),
  PRIMARY KEY  (pname, ssn),
  FOREIGN KEY  (ssn) REFERENCES Employees
    ON DELETE CASCADE
    ON UPDATE CASCADE);
```

Review: ISA Hierarchies



Translating ISA Hierarchies to Relations

- Create tables for both parent and child entities
  - *Employees*: (*ssn*, *name*, *lot*)
  - *Hourly_Emps*: (*ssn*, *hourly_wages*, *hours_worked)*
    - *ssn* both primary and foreign key in Hourly_Emps!
    - Must delete Hourly_Emps tuple if referenced Employees tuple is deleted.
- Create tables only for child entities
  - *Hourly_Emps*: (*ssn*, *name, lot, hourly_wages, hours_worked)*
  - Each employee must be in one of these two subclasses.

Summary of Conceptual Design

- *Conceptual design* follows *requirements analysis*
  - Yields a high-level description of data to be stored
- ER model popular for conceptual design
- Fundamental constructs: *entities*, *relationships*, and *attributes* (of entities and relationships).
- Additional constructs: *weak entities*, *ISA hierarchies*, and *aggregation* (not seen here).
- *Constraints* are important elements of a conceptual model. They reflect real-world aspects of its functioning.

Summary of ER (Contd.)

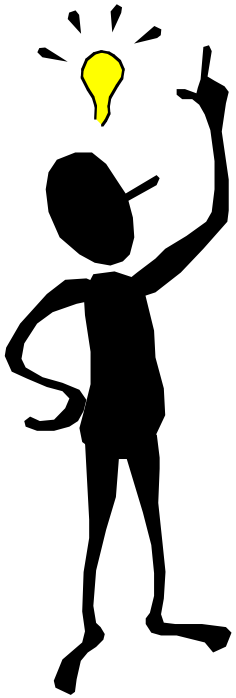
- ER design is *subjective*. There are often many ways to model a given scenario! Analyzing alternatives can be tricky, especially in a complex setting.

- Ensuring good database design: resulting relational schema should be a*nalyzed and refined further*.
  - Functional dependency information and normalization techniques (to be seen)

Relational Model: Summary

- A tabular representation of data.
- Simple and intuitive, currently the most widely used.
- Integrity constraints can be specified by the DBA, based on application semantics. DBMS checks for violations.
  - Two important ICs: **primary** and **foreign keys**
  - In addition, we *always* have domain constraints.
- Powerful and natural query languages exist.
- Rules to translate ER to relational model

Questions



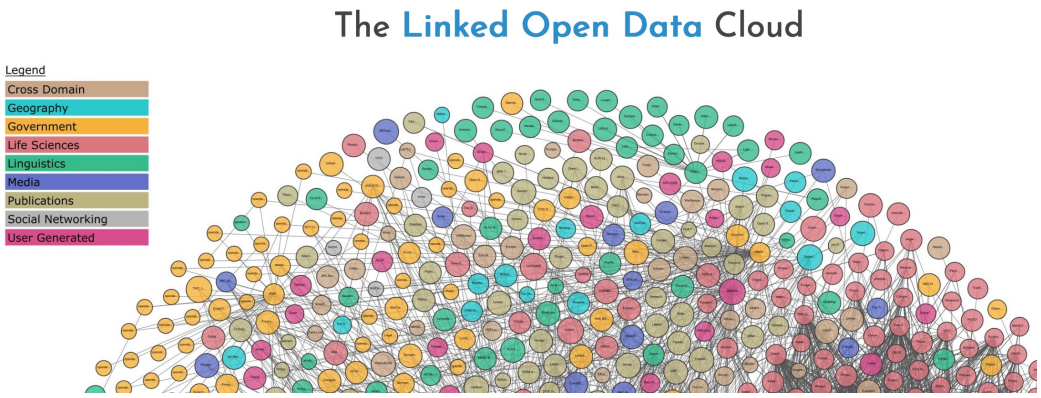
Beyond tables: graph databases

- The relational data model can be seen as too rigid
  - If one book has an extra attribute, e.g., editorComment, the whole table needs a new column; most of its values will be empty.
- The schema and data are rigidly separated
  - Impossible to ask for "all the tables having an attribute price with at least three values under 50"
- Hard or impossible to ask for connections if one does not know what the connection pattern
  - E.g., *how* is Harry *connected* to Sally in this database?

A model for graph databases (1): RDF

- Resource Description Framework (World Wide Web Consortium)
- Each piece of data **(triple)** states a value for a property that a resource (subject) has: (subject, property, object)
- An RDF database, or **RDF graph**, is a set of triples.
- Each subject is an International Resource Identifier (or IRI, standardized format). Previously, Universal Resource Identifiers (URIs).
- All properties are also IRIs. Values may also be IRIs, or literals

https://lod-cloud.net



https://www.wikidata.org/wiki/Q273626

WIKIDATA Search Wikidata Search

École polytechnique (Q273626)

Item Discussion Read

French engineering grande école (selective higher education institution)

Polytechnique | L'École Polytechnique | Ecole polytechnique | École polytechnique (France) | L'X | Ecole polytechnique (Paris)

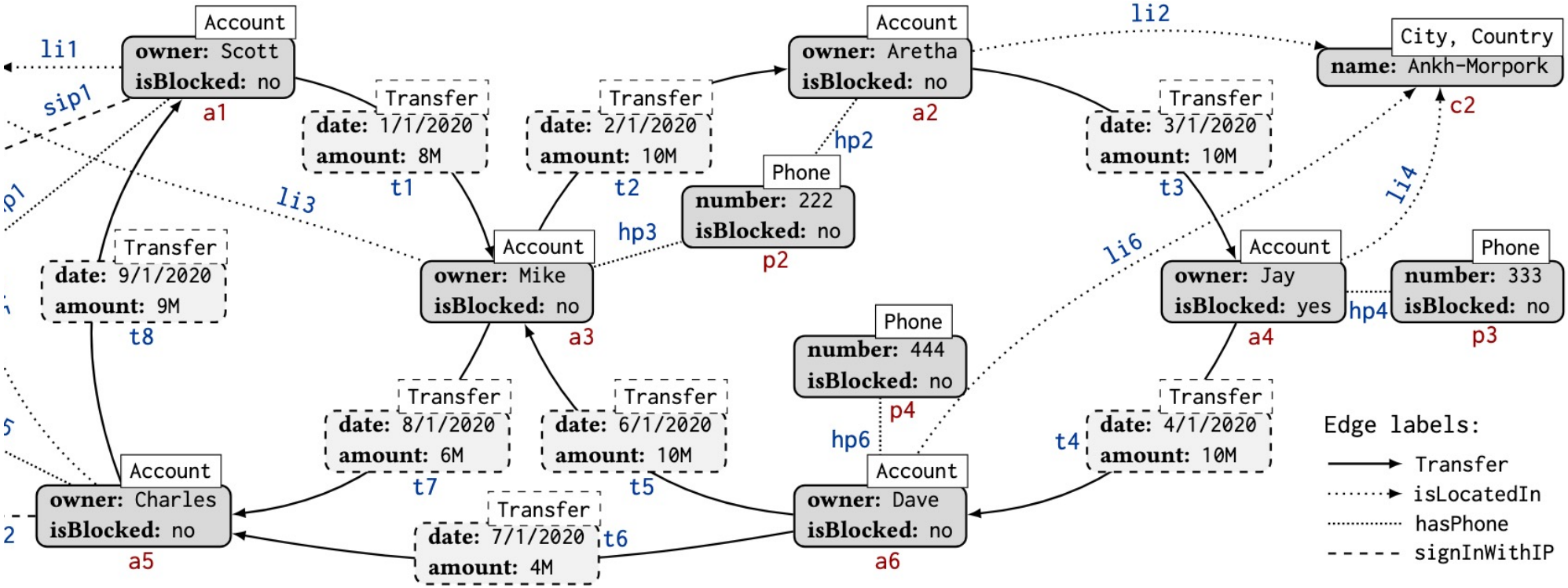
In more languages

Configure

Language	Label	Description	Also known as
default for all languages	No label defined	–	
English	École polytechnique	French engineering grande école (selective higher education institution)	Polytechnique L'École Polytechnique Ecole polytechnique École polytechnique (France) L'X Ecole polytechnique (Paris)
French	École polytechnique	grande école d'ingénieurs publique française	Polytechnique École centrale des travaux publics Ecole polytechnique X l'X

A model for graph databases (2): Property graphs

- Nodes resemble Entities from the E-R conceptual modeling, except that:
  - Each node may have one or several labels
- Relationships may also have attributes
- Like in RDF graphs, no fixed set of attributes or labels!



From https://dl.acm.org/doi/10.1145/3514221.3526057

Graph vs. relational databases

Relational databases

- Since 1970's
- Some *rigidity*; requires advance knowledge of the data structure (schema)
- Efficient, complex systems underlying any major public or private organization today

Graph databases

- Since 2000 (with some earlier attempts, e.g., object-oriented in 1990's)
- Support high levels of *heterogeneity*
- Performance at scale still object of research